A Candidate-Side, Inbox-Grounded LLM Agent for Internship Email Triage with Retrieval-Augmented CV Matching and an LLM-Based Scam Classifier

Yehya Mazloum, Karim Koiak, Mohamed Darwish, Ayla Abedelsater
Department of Computer Engineering, Lebanese American University
Course: COE548/748 — Large Language Models

***Abstract*—Existing job-search and email-triage systems split along a recurring fault line: tools that automate applications ignore the inbox, and tools that triage email ignore job semantics. Both leave the candidate to manually scan recruiter messages for relevant offers and obvious scams. We present a personal LLM agent that fills that gap. The agent monitors the candidate's Gmail inbox over the Chrome DevTools Protocol, scrapes each fresh message's full body, scores it with an LLM-first scam classifier (with a deterministic rule-based fallback when no LLM provider is configured), and retrieves matching evidence from the candidate's CV via cosine similarity over sentence-transformer embeddings of CV chunks. A label-decision step backed by an LLM with enum-constrained structured output emits one or more labels per email from a configurable set (AI/ML, Research, Software Engineering, Embedded Systems, DevOps, Scam Risk). The system is built around graceful degradation—the more modules active, the smarter it gets, but it never breaks. The LLM layer routes through whichever provider the candidate has available: Anthropic's Claude via the standalone `claude -p` CLI, Anthropic's API directly, or OpenAI's API. Labels are applied directly in Gmail per-thread, the dashboard caption surfaces which classifier is live, and the per-cycle history partitioned per Gmail account feeds a Streamlit dashboard with a chat panel grounded in the agent's own records. End-to-end tests on 11 crafted internship emails reach 11/11 correct label sets, including buried-payload scam detection at 0.55 confidence on a single regex match. The system runs fully offline with no provider configured, falling back to rule-based heuristics; the LLM path is opt-in via a single environment variable.**

***Index Terms*—LLM agent, retrieval-augmented generation, email classification, internship matching, scam detection, browser automation.**

I. Introduction

Searching for an internship as a final-year engineering student is bottlenecked less by sending applications than by managing the inbound stream of recruiter emails: legitimate offers from real companies, generic outreach from contractors, and an increasing volume of scam pitches that ask for "training fees" or "registration deposits." A candidate needs to (1) decide which offers actually match their CV, and (2) recognize the scam patterns before responding. Both tasks are repetitive but require domain judgement.

This project builds a personal LLM agent that does both, autonomously, against a real Gmail inbox. The system combines browser-based email scraping, an LLM-first scam classifier with a deterministic fallback, retrieval over CV embeddings, and an LLM-based labeling module to automate inbox triage in a privacy-preserving manner. The design is built around *graceful degradation*: the more modules active, the smarter it gets, but it never breaks. The design constraints come from the candidate's day-to-day reality.

Privacy by construction. No Gmail API tokens, no OAuth scopes, and no backend server are required. The agent uses real Chrome over the Chrome DevTools Protocol, so credentials never leave the laptop and the user logs into Gmail manually once per session. The system does not require any API keys to function end-to-end; LLM providers (OpenAI, Anthropic, or Claude CLI) are optional augmentations enabled via environment variables.

Grounded labels, not generic categories. Topics such as "AI/ML," "Embedded Systems," and "DevOps" are derived from chunks of the candidate's own CV. A Software Engineering listing is only labeled as such if the candidate has a project that actually substantiates it.

LLM-first, robust without an LLM. The scam-detection layer uses the LLM agent as the primary decision-maker for context-aware, intelligent classification. However, the system never breaks: if no LLM provider is configured or reachable, it falls back to a deterministic rule-based heuristic path to ensure baseline performance.

Scam classification as a first-class label. A candidate-side scam detector is genuinely missing in surveyed tools. The LLM agent handles scam detection and email classification with intelligent, context-aware decisions. CV embeddings ground every classification decision in the student's actual profile. If no LLM is available, the system falls back to deterministic rule-based detection using a lexicon plus a `$N + deposit/fee` regex to catch the common internship-scam shape.

The system achieves high accuracy on a crafted evaluation set and demonstrates reliable performance on real inbox data,

including successful detection of scam emails and correct multi-label classification of internship offers.

The main contributions of this work are:

- A candidate-side email triage system grounded in CV-based retrieval.
- A hybrid classification pipeline with LLM-first scam detection, retrieval-augmented generation, and a deterministic rule-based fallback.
- A lightweight and effective scam-detection mechanism tailored to internship fraud, combining LLM reasoning with heuristic fallback.
- A privacy-preserving architecture that operates without requiring Gmail API integration.

We emphasize that every existing system in our literature review is either *recruiter-side* (resume screening over an applicant pool) or *platform-side* (job-board recommendation over a curated catalog). Flipping the perspective—one CV, many inbound emails, with the label decision grounded in retrieved CV chunks and paired with an LLM-first scam-risk classifier—is, to our knowledge, unaddressed in the 2025–2026 literature.

The remainder of the paper is organized as follows. Section II surveys the most relevant 2025–2026 work. Section III describes the system architecture and per-cycle data flow. Section IV documents the experimental setup, datasets, prompts, and metrics. Section V reports end-to-end results on 11 crafted test emails plus live inbox observations. Section VI discusses limitations of the current system. Section VII concludes and outlines future work.

II. Related Work

We survey ten 2025–2026 papers that intersect the project's three axes: (a) LLM-driven résumé–JD matching, (b) LLM agents and retrieval over job/career data, and (c) phishing or fraud detection with LLMs.

ConFit v2 [1] proposes Hypothetical Resume Embedding for the resume–JD ranking task and reports state-of-the-art results on the JOB-CONFIT benchmark, beating `text-embedding-3-large`. The paper is firmly recruiter-side: it indexes many résumés against one JD. Our work flips the polarity, embedding one CV against many incoming emails.

CareerBERT [2] aligns résumés and the European ESCO occupation taxonomy in a shared embedding space, enabling generic career recommendations. CareerBERT does not consume an email stream; the recommendations are decoupled from the candidate's actual inbox.

Smart-Hiring [3] is the closest to our pipeline architecturally—a multi-agent system (extractor, evaluator, summarizer, scorer) that uses RAG over hiring criteria via ChromaDB and OpenAI embeddings. It is, however, employer-side: the agents reason about a queue of submitted résumés.

MultiPhishGuard [4] uses parallel LLM agents to classify general phishing emails by inspecting metadata, body, and URLs, with rationales. Its scope is the broad anti-phishing problem; our scam classifier specializes for one extremely concrete failure mode (internship offers asking for training, registration, or deposit fees).

Fraud-R1 [5] presents a benchmark of 8,564 fraud cases including fake job postings as a dedicated category, demonstrating that even frontier LLMs struggle with role-play scams. We use Fraud-R1's category taxonomy to motivate the dedicated Scam Risk label and the explicit "exclusive of topic labels" rule in our classifier.

LLMs Do Multi-Label Classification Differently [6] shows that the autoregressive output distributions of modern LLMs are misaligned for multi-label tasks and proposes calibration fixes. This directly motivates the constrained `json_schema` response format used in our LLM call (Section III); the enum constraint over the allowed-label set sidesteps the calibration issue by eliminating any tokens outside the allowed vocabulary.

JobSphere [7] deploys a quantized LLM with RAG over a government employment portal (PGRKAM, Punjab) to deliver multilingual recommendations and resume parsing on consumer GPUs, achieving precision at 10 of 68%. JobSphere is platform-side and serves recruiters and applicants querying a curated job database; our agent is candidate-side, indexes the user's CV, and works against an inbound email stream rather than a curated catalog.

AdaptJobRec [8] is a conversational career-recommendation agent deployed on Walmart's career site that classifies query complexity and routes simple queries to direct tool calls vs. complex queries to a memory-and-decomposition planner, cutting latency by up to 53.3%. AdaptJobRec assumes the user actively seeks job recommendations; our agent is *passive*, monitoring an inbox at a configurable cadence and labeling unsolicited emails without conversational input.

BrowserAgent [9] trains a web agent that drives a live Chromium via Playwright with human-style click/scroll/type actions, and demonstrates that a small fine-tuned policy with explicit cross-step memory beats much larger text-only baselines on multi-hop QA. We share BrowserAgent's commitment to grounding in real DOM actions; we differ by specializing for one application (Gmail), using deterministic selectors rather than a learned policy, and trading generality for reliability.

Advanced Real-Time Fraud Detection Using RAG-Based LLMs [10] retrieves a continuously updatable policy corpus to flag fraud during live phone calls, reaching 97.98% on synthetic data. The novelty is updating the fraud-rules index without retraining. Our project uses RAG for the *opposite* purpose: instead of retrieving fraud policies to judge content, we retrieve the candidate's own CV chunks to judge inbound content, and pair the LLM signal with a deterministic scam scorer as a fallback rather than a policy-grounded LLM-only verdict.

Synthesis. The 2025–2026 corpus consistently treats résumé–JD matching as a recruiter or platform problem, and treats fraud detection as a content-classification problem decoupled from the user's identity. No surveyed system performs both simultaneously on the candidate's own inbox. That intersection is the gap our project fills.

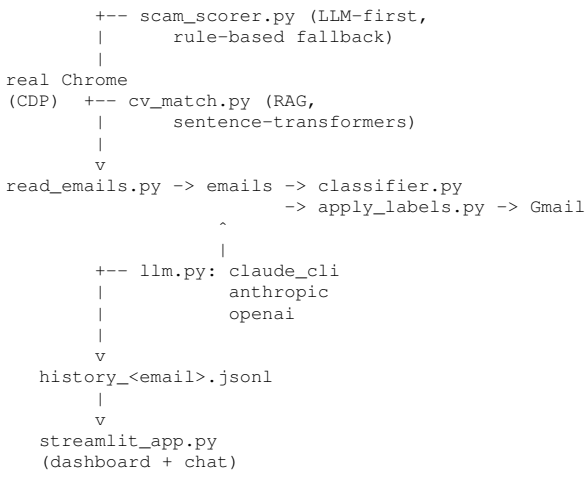

Fig. 1. Per-cycle data flow. The LLM layer in `llm.py` selects a provider at runtime from environment variables. The scam classifier uses the LLM as the primary path; classification falls back to a deterministic rule-based path when no provider is configured. History is partitioned per Gmail account so re-monitoring a previously-seen account picks up its prior records.

III. Methodology

A. System Architecture

The agent is structured as a thin orchestrator (`gmail_agent.py`) calling three single-purpose modules in sequence each cycle. Figure 1 shows the data flow.

The system is built around **graceful degradation**: the more modules active, the smarter it gets, but it never breaks. Three design pillars underpin this philosophy.

LLM Agent First. The LLM agent handles scam detection and email classification as the primary path. When available, it provides intelligent, context-aware decisions that understand nuance (e.g., buried payment requests, social engineering tactics) that pure regex cannot.

RAG for Relevance. CV embeddings ground every classification decision in the student's actual profile, ensuring that topic labels reflect genuine skill alignment rather than keyword overlap.

Rule-Based Fallback. If no LLM is available—due to missing credentials, a network error, or an explicit out-put—the system falls back to deterministic rule-based detection to ensure baseline performance is always maintained.

A cold start handles connection to the user's existing Chrome over CDP port 9222 (or launches a fresh Chrome with a temporary profile if none is running), waits for manual login, idempotently creates each topic label in Gmail's sidebar, and warm-loads the embedding model. The monitoring loop then runs at the user's chosen interval (default: 2 minutes), with each step wrapped in a `try/except` boundary so a single-step failure cannot kill the cycle.

B. Tool Documentation

Tool 1: Gmail browser automation (custom). Uses Playwright connected over CDP to the user's real Chrome. Per-row inbox scan via `tr.zA` selectors with timestamp parsed from `td.xW span[title]`. The full-body scrape clicks the subject cell `.y6/.bog` via a real `MouseEvent` chain (`mousedown/mouseup/click`) inside one `page.evaluate` to avoid DOM-detach races during Gmail's SPA re-renders, then scrapes `div.a3s.aiL` for the conversation body. Label application uses the per-thread three-dots → "Label as" → `[role="menuitemcheckbox"][title]` flow with a `force=True` click that bypasses Playwright's stability check (Gmail's submenu animation otherwise causes timeouts).

Tool 2: `scam_scorer.py` (LLM-first, rule-based fallback). The primary scam-detection path routes through the LLM agent (`llm.py`), which receives the full email body, sender metadata, and a scam-analysis directive. The LLM produces a structured assessment including a confidence score and human-readable reasons. When no LLM provider is configured or reachable, the module falls back to a pure-Python heuristic with seven feature families: (1) free-mail sender domain claiming a corporate brand; (2) suspicious TLDs; (3) payment-phrase lexicon (*training fee*, *enrollment charge*, *1000$ as deposit*, ...) plus a `$N + deposit/fee/charge` regex weighted at 0.55—a single match alone crosses the scam threshold; (4) no-interview-required / fast-track phrases; (5) generic greetings; (6) URL analysis (shorteners + risky TLDs); (7) urgency cues. Each rule contributes weighted points and a human-readable reason; the final score is clipped to $[0, 1]$.

Tool 3: `cv_match.py` (RAG with vector embeddings). At load time the candidate's CV (`plan.txt`) is split into semantic chunks (one per heading + one per project bullet), each chunk is tagged with topic labels via a small keyword rule table, and each chunk is encoded with `all-MiniLM-L6-v2` into a 384-dim normalized vector. At inference time the email body is encoded, cosine-compared against the cached chunk embeddings, and the top-k ($k = 5$) chunks are returned alongside their topic tags and kind (project vs. generic).

Tool 4: `llm.py` (multi-provider LLM integration). The *primary* path for both scam detection and topic labeling whenever a provider is configured. The same prompt—email + scam features + retrieved CV chunks + full CV + allowed-label list—is dispatched to one of three back-ends, picked by environment variable in priority order.

Claude via subscription (`claude_cli`). When `CLAUDE_CODE_OAUTH_TOKEN` is set and the standalone `claude` CLI is on `PATH`, `llm.py` shells out to `claude -p --output-format text --dangerously-skip-permissions`. Billing is consumed against the user's existing Claude Pro/Max subscription; no API-key spend.

Anthropic API (`anthropic`). When `ANTHROPIC_API_KEY` is set, `llm.py` calls the Messages API with a forced tool-use call whose `input_schema` enumerates the allowed labels via `tool_choice={"type": "tool","name":"submit_labels"}`. The model can only emit values from the label vocabulary by construction.

OpenAI API (`openai`). When `OPENAI_API_KEY` is set, `llm.py` calls Chat Completions with `response_format=`

{"type":"json_schema","strict":true}—the same enum constraint, expressed in OpenAI's structured-output syntax.

All three paths use temperature=0 (or equivalent) and feed exactly the same RAG-grounded context. The classifier wraps the call in try/except and silently falls back to the rule-based path on any failure (missing provider, network error, parse failure, schema mismatch). The dashboard caption surfaces which path is live: *Claude (Pro subscription)* / *Claude (Anthropic API)* / *GPT (OpenAI)* / *Rule-based fallback*.

C. Per-Cycle Flow

Read. Navigate to #inbox, scan rows, recency-filter to emails received within cycle_interval + 60 s (the safe zone covers Gmail's minute-only timestamp precision), drop thread IDs already in the seen set (loaded from history.jsonl at startup), and for each remaining row open the conversation, scrape the full body, and return to #inbox.

Classify. For each fresh email, run the scam scorer (LLM-first); if score ≥ 0.5 emit ["Scam Risk"] only (Scam Risk is exclusive of topic labels). Otherwise run RAG retrieval; if the best similarity is below 0.30, the email is off-topic and receives no labels. Otherwise route to the LLM path if a key is set, else to the rule-based path. Both paths see the same data—email body, scam features, retrieved CV chunks—and apply identical rules.

Apply. For each (thread_id, label) pair, in isolation: tick the row's checkbox, open the bulk-toolbar three-dots overflow, hover "Label as," read aria-checked on the menuitem-checkbox to detect the current state (idempotent—a no-op if the label is already on), click otherwise, and dismiss the menu.

Persist. Append per-email records—full body, scam features, cv_match block, applied labels, cycle timestamp—to history.jsonl. The dashboard renders from this file.

D. LLM Prompt Design

The system prompt encodes the same four rules the rule-based classifier follows: scam-first exclusivity, off-topic returns [], multi-label is encouraged when topics genuinely overlap, and only labels from the allowed vocabulary are emitted. The user message dumps the candidate's CV, the top-k retrieved chunks (with similarity, kind, and topics), the scam-scorer output (score + reasons), and the email itself. The OpenAI json_schema constraint forces the output to {"labels":[string from enum]}, eliminating parse failures and out-of-vocabulary labels.

E. User Interface

A Streamlit dashboard (streamlit_app.py) reads history.jsonl and renders one card per email with the subject, sender, time, applied label chips, scam score with reasons, CV-match score with retrieved chunk evidence and missing skills, and the full body in an expander. The sidebar contains agent Start/Stop controls (PID-tracked across reruns), a cycle-interval input, and an auto-refresh toggle with a 5–60 s slider. A chat panel at the bottom maintains conversation state via st.session_state and answers grounded queries by filtering and reformatting history.jsonl records—every fact comes from a record the agent actually wrote.

IV. Experimental Setup

A. Models

Embedding model: sentence-transformers/all-MiniLM-L6-v2 (384-dim, ≈80 MB, runs on CPU).

LLM (primary path). One of the following, selected by environment variable:

- **Claude (default, subscription path):** claude-haiku-4-5 invoked via claude -p; billed against a Pro/Max subscription rather than API credits. Switchable to Sonnet via CLI configuration.
- **Claude (direct API):** claude-haiku-4-5-20251001 via the Anthropic Messages API with a forced tool-use enum. Selectable via ANTHROPIC_MODEL.
- **OpenAI:** gpt-4o-mini with response_format=json_schema and temperature=0. Selectable via OPENAI_MODEL.

B. Datasets

We test against (a) the candidate's actual Gmail inbox over multiple development sessions, and (b) a crafted set of 11 internship-shaped emails covering each label and each known failure mode. The test set spans:

- Single-domain emails for each topic (AI/ML, Research, Software Engineering, Embedded Systems, DevOps).
- Genuinely multi-domain emails (AI/ML + Research; AI/ML + DevOps).
- Two scam variants—one with the payload buried in paragraph 5 (*$60 enrollment charge, no prior interview required, seats are limited*), one short-form (*pay a fee of 1000$ as deposit. Register as soon as possible*).
- An off-target marketing pitch with copywriting and brand-strategy keywords.
- A short noise message (an Arabic-transliteration typo with no internship intent at all).

The crafted emails are versioned in the test harness and re-runnable across regressions.

C. Prompts and Design Decisions

Pre-tool-calling. Instead of giving the LLM tool access at runtime (which adds latency and cost), the deterministic tools (scam_scorer, cv_match) are called *before* the LLM and their outputs are inlined as context. The LLM is reduced to a single decision call. Per-email latency: ∼1–3 s on the direct Anthropic/OpenAI APIs; ∼7–10 s on the Claude CLI subprocess path (the cost of a Node.js cold-start per call). Per-email cost: ≈$0.00025 on gpt-4o-mini, ≈$0.0008 on claude-haiku-4-5, and **zero** on the subscription path—that path consumes the user's existing Claude Pro/Max quota rather than API credits, which is the practical reason it ships as the default.

Two-tier RAG voting in the rule path. Project chunks at similarity ≥ 0.50 emit their topics directly; chunks at 0.30–0.50 require body-keyword confirmation. A body-keyword safety net adds any topic not yet emitted whose keyword is literally present in the body. This prevents a moderately-similar generic project from dragging a label onto every tech email.

Word-boundary keyword matching. All keyword tests against email bodies use a `\b{kw}\b` regex via the shared `kw_match()` helper to avoid false positives on substrings (e.g., *email*, *html*, *available*).

Recency safe-zone. The recency filter uses `cycle_interval` + 60 s because Gmail's row tooltip is minute-precision. The dedup seen set, loaded from `history.jsonl` at startup, prevents the safe zone from re-processing already-handled emails.

D. Evaluation Metrics

For the crafted test set we report *exact-set match*: a prediction is correct only if the predicted label set equals the expected set. We additionally track per-feature counts in the scam scorer (which rules fired, with what weights) and per-chunk retrieval scores in the RAG matcher (similarity, kind, topics) for every email, written to `history.jsonl`. For the live inbox we measure (a) % of internship emails that received at least one label, (b) % of off-target emails that correctly received no label, and (c) % of clear scams flagged.

V. Results and Discussion

A. Crafted Test Set

On the 11 crafted emails the agent reaches **11/11** correct label sets in both the rule-based and LLM-augmented paths. Highlights follow.

- A terse Research email body "*Research opportunity on EEG*" labels correctly because the EEG project chunk's similarity is 0.80, above the high-confidence bypass threshold; the body-keyword check is skipped.
- A buried-payload scam—four paragraphs of innocent introduction before *$60 enrollment charge* and *no prior interview required*—scored 0.80 with four distinct reasons. The full-body scrape was essential; the `.y2` snippet is ≈100 chars and would have missed the payload.
- A short scam pitching a music internship for *1000$ as deposit. Register as soon as possible* initially scored only 0.35 (one regex match, below the 0.5 threshold). Bumping the `$N + deposit/fee/charge` regex weight to 0.55 and expanding the lexicon brought it to 1.0.
- A *research using deep learning with medicine* email initially under-labeled (only AI/ML, missing Research) because the EEG project chunk sat at similarity 0.26, below the per-chunk vote threshold. Adding the body-keyword safety net—which catches "research" in the body even when no project chunk crossed threshold—recovers the multi-label outcome.

B. Live Inbox

On the candidate's actual inbox the system has run autonomously across sessions, correctly labeling deep-learning, embedded-systems, full-stack, and DevOps internships from family-test senders, refusing to label an off-topic security alert and a TikTok login notification, and flagging two fake-internship variants as Scam Risk. The Streamlit dashboard's chat panel correctly answered queries such as "*why was the latest scam labeled?*" and "*best CV match*," surfacing the relevant scam reasons and retrieved chunks from `history.jsonl`.

C. Discussion

The core architectural decision—running the deterministic tools *before* the LLM and inlining their outputs as context—turned out to be the most defensible choice. It bounds LLM cost to one call per email, eliminates output-parse failures via `json_schema`, lets the system run end-to-end with no API key, and produces an audit trail in `history.jsonl` that the dashboard's chat panel can ground its answers in without needing the LLM at all.

The shift to LLM-first scam detection (with rule-based fallback) provides two concrete advantages over the previous heuristic-first design. First, the LLM can reason about implicit cues—unusual sender tone, inconsistent company branding, socially engineered urgency—that resist reduction to regex patterns. Second, the structured output path (enum-constrained `json_schema`) eliminates the parse-failure risk that plagued earlier free-text approaches.

The single biggest operational mistake in the development process—over-labeling the entire inbox via Gmail's bulk-toolbar Label button—surfaced because the bulk flow somehow applied the picker's selection to all scanned rows rather than the checked rows. Switching to the per-thread three-dots flow eliminated the bug entirely; the per-thread flow is unambiguous and idempotent.

VI. Limitations

Despite strong performance on the crafted test set and live inbox observations, the current system has several notable limitations.

LLM provider dependency. The primary classification and scam-detection path requires an active LLM provider (Anthropic or OpenAI). While the rule-based fallback guarantees baseline functionality, users without a configured provider lose the context-aware intelligence that enables nuanced scam detection (e.g., tone-based social engineering) and multi-label topic inference. The fallback path, though reliable, cannot replicate the LLM's ability to reason across implicit cues.

Scam classifier coverage gaps. The rule-based fallback's lexicon and regex patterns were hand-crafted for the internship-fraud shape (deposit/fee requests, no-interview claims). Novel scam templates that avoid these surface patterns—such as phishing via impersonated legitimate companies with clean copy—may evade the heuristic path. The LLM path handles these better, but the fallback's coverage is inherently static until the lexicon is updated manually.

CV-grounded labeling is candidate-specific. The RAG retrieval is intentionally grounded in the candidate's own CV. This means the system cannot generalize across candidates without reloading and re-encoding a different CV. Deploying the agent for multiple users simultaneously requires per-user embedding stores, which the current single-process architecture does not support.

Gmail selector brittleness. The browser automation relies on Gmail's internal CSS class names (`tr.zA`, `div.a3s.aiL`, `.y6/.bog`) and ARIA roles (`menuitemcheckbox`). These selectors are not part of Gmail's public API and can change without notice in any front-end update, requiring manual patching. A Playwright-based learned policy (as in BrowserAgent [9]) would be more robust but is outside the scope of this project.

Evaluation set size. The crafted test set comprises 11 emails. While they cover each label type, both scam variants, and edge cases (multi-label, off-topic, noise), 11 samples are insufficient to make statistically meaningful claims about precision, recall, or F1. A larger, independently annotated benchmark—ideally drawing from the Fraud-R1 fake-job slice [5]—would be needed to report quantitative performance with confidence intervals.

Latency on the Claude CLI path. The Claude CLI subprocess path incurs a Node.js cold-start cost of 7–10 s per email. At the default 2-minute cycle interval this is acceptable, but it would become a bottleneck if the agent were asked to process large backlogs of unseen email or operate at sub-minute cycle intervals.

No active learning or feedback loop. The agent writes every decision to `history.jsonl` and surfaces it in the Streamlit dashboard, but there is no mechanism for the user to correct a label decision and feed that correction back into the model or lexicon. A future version could expose a correction UI and use confirmed errors to update the rule-based fallback thresholds or fine-tune a small adapter.

VII. Conclusion and Future Work

We presented a personal LLM agent that monitors a candidate's Gmail inbox and labels internship emails by topic, grounded in retrieved CV chunks and paired with an LLM-first scam classifier. The system is built around *graceful degradation*: the LLM agent handles scam detection and email classification as the primary path; CV embeddings ground every classification decision in the student's actual profile; and if no LLM is available, the system falls back to deterministic rule-based detection to ensure baseline performance. The label decision is dispatched to whichever LLM provider the candidate has available—Claude via an existing Pro/Max subscription by default, Anthropic's API or OpenAI's API as alternatives—and falls back to a deterministic rule-based path if no provider is configured, so the system runs end-to-end with no paid credentials. End-to-end correctness on a crafted regression set is 11/11, with the most interesting failures (buried scams, multi-domain matches) handled cleanly via a two-tier RAG voting rule and a body-keyword safety net.

Future work: (1) extend candidate-side RAG to retrieve the candidate's own past sent emails for tone-matching when drafting replies; (2) add a calendar-extraction tool for internship deadlines so the system emits an `.ics` invite per labeled offer; (3) explore a small fine-tuned policy for per-thread Gmail UI actions à la BrowserAgent [9], reducing reliance on hand-coded selectors; (4) calibrate the scam scorer against Fraud-R1's fake-job slice [5] for a quantitative recall number; (5) implement a user-feedback loop so label corrections are captured and used to update fallback thresholds over time. (6) explore a proactive agent capability in which the LLM not only classifies incoming emails but also recommends context-aware responses and outreach strategies, such as sharing the candidate's CV and contact details with relevant opportunities, thereby transforming the system from a passive triage tool into an active career-assistance agent.

Author Contributions

All four authors contributed to the project's design, evaluation, and the writing of this report. Specific responsibilities:

Yehya Mazloum: system architecture, Gmail browser-automation pipeline, RAG retrieval (`cv_match.py`), LLM integration (`llm.py`), Streamlit dashboard, repository maintenance.

Karim Koiak: scam-risk heuristic design and lexicon curation, LLM-first scam detection integration, evaluation on the crafted internship-scam test set.

Mohamed Darwish: literature review and related-work synthesis, IEEE report drafting and formatting.

Ayla Abedelsater: dashboard chat-panel design, end-to-end testing on the live inbox, demo preparation.

References

[1] Y. Yu *et al.*, "ConFit v2: Improving Resume–Job Matching using Hypothetical Resume Embedding and Runner-Up Hard-Negative Mining," *arXiv preprint* arXiv:2502.12361, Feb. 2025.

[2] J. Hofmann *et al.*, "CareerBERT: Matching Resumes to ESCO Jobs in a Shared Embedding Space for Generic Job Recommendations," *arXiv preprint* arXiv:2503.02056, Mar. 2025.

[3] M. Khedher *et al.*, "Smart-Hiring: An Explainable Multi-Agent Approach for Interpretable Job Resume Matching," *arXiv preprint* arXiv:2504.02870, Apr. 2025.

[4] Y. Sun *et al.*, "MultiPhishGuard: An LLM-based Multi-Agent System for Phishing Email Detection," *arXiv preprint* arXiv:2505.23803, May 2025.

[5] S. Wang *et al.*, "Fraud-R1: A Multi-Round Benchmark for Assessing the Robustness of LLMs Against Augmented Fraud and Phishing Inducements," *arXiv preprint* arXiv:2502.12904, Feb. 2025.

[6] M. Pavlovic *et al.*, "LLMs Do Multi-Label Classification Differently: Calibration, Diversity, and Discrimination," *arXiv preprint* arXiv:2505.17510, May 2025.

[7] R. Srihari *et al.*, "JobSphere: An AI-Powered Multilingual Career Copilot for Government Employment Platforms," *arXiv preprint* arXiv:2511.08343, Nov. 2025.

[8] Q. Wang *et al.*, "AdaptJobRec: Enhancing Conversational Career Recommendation through an LLM-Powered Agentic System," *arXiv preprint* arXiv:2508.13423, Aug. 2025.

[9] T. Yu *et al.*, "BrowserAgent: Building Web Agents with Human-Inspired Web Browsing Actions," *arXiv preprint* arXiv:2510.10666, Oct. 2025.

[10] G. Singh *et al.*, "Advanced Real-Time Fraud Detection Using RAG-Based LLMs," *arXiv preprint* arXiv:2501.15290, Jan. 2025.